

# Manual técnico de desarrollo modular

Sistema web y app operativa para estética canina, SPA y hotel · Guía base para construir en Google Antigravity

Este documento define la arquitectura, estructura de carpetas, reglas de modularidad, flujo de trabajo en Antigravity, estándares de código y lineamientos de base de datos para desarrollar un sistema escalable. La meta es separar claramente marketing, operación, datos y automatizaciones, evitando mezclar lógica de negocio con vistas, estilos o scripts sueltos.

| Versión | Proyecto                      | Ámbito                                   | Fecha      |
|---------|-------------------------------|------------------------------------------|------------|
| v1.0    | Estética canina / SPA / Hotel | Web pública + backoffice + API + Android | Marzo 2026 |

**Decisión principal: WordPress se usa para marketing y captación; Laravel/API y la base de datos propia son el núcleo operativo. La app Android y el panel administrativo consumen la misma API para evitar duplicidad de lógica.**

## 1. Objetivo del manual

Definir una forma de trabajo repetible para construir el sistema de manera modular, auditable y mantenible. El documento cubre la organización del repositorio, los módulos funcionales, la separación por capas, los estándares para HTML, CSS, JavaScript y PHP, la estrategia de base de datos, el consumo de API y la forma correcta de delegar tareas en Antigravity.

- Evitar archivos mixtos con HTML, CSS, JS, SQL y lógica de negocio en el mismo lugar.
- Preparar el sistema para crecer desde un MVP de agenda hasta operación completa con hotel, ocupación real, estados de servicio, leads y recordatorios.
- Permitir que Antigravity trabaje por tareas pequeñas, verificables y con artefactos claros.
- Asegurar compatibilidad con SEO, panel responsivo y futura app Android.
- Mantener el diseño preparado para automatizaciones y notificaciones posteriores sin rehacer la arquitectura.

## 2. Principios de arquitectura

- Separación de contextos: sitio público, backoffice, API, base de datos y notificaciones se diseñan como dominios distintos.
- Una sola fuente de verdad: clientes, mascotas, reservas, ocupación y estados viven en la base de datos central, no en hojas sueltas ni en plugins aislados.
- Módulos por negocio: leads, clientes, mascotas, citas SPA, hotel, ocupación real, estados, notificaciones y reportes.
- Lógica en servicios: controladores delgados; la lógica de negocio vive en clases de servicio y repositorios.
- UI desacoplada: las vistas no hacen consultas complejas ni contienen reglas de negocio.
- API primero: todo lo que use la app Android o un panel móvil debe pasar por endpoints versionados.

- Preparación para escalar: primero manual y estable; después automatización, tiempo real, mensajería y dashboards.

### 3. Antigravity como entorno de desarrollo

Antigravity se aprovecha mejor cuando las tareas se formulan como entregables concretos: plan, implementación, diff, pruebas, captura visual y checklist de validación. Para este proyecto conviene usar Antigravity como orquestador de tareas por módulo, no como generador indiscriminado de todo el sistema en un solo paso.

#### 3.1 Reglas de trabajo en Antigravity

- Usar modo Planning para tareas complejas: arquitectura, refactorización, diseño de base de datos, pruebas o integración entre módulos.
- Usar modo Fast para tareas localizadas: renombrar archivos, ajustar estilos, mover componentes o arreglar validaciones puntuales.
- Pedir siempre artefactos verificables: plan técnico, árbol de archivos afectados, endpoints, diagrama del flujo y evidencia visual.
- Configurar política de revisión para que cualquier comando riesgoso, migración destructiva o cambio masivo requiera revisión humana.
- Activar lista de URLs permitidas para el navegador del agente cuando se consulte documentación externa o se prueben paneles internos.
- Usar MCP para conectar herramientas locales, bases de datos, terminal y servicios auxiliares cuando el flujo lo requiera.

#### 3.2 Convención de prompts de trabajo

Contexto:  
 Proyecto de estética canina / SPA / hotel.  
 Stack: WordPress público + Laravel backoffice/API + PostgreSQL.  
 Objetivo de esta tarea:  
 Crear el módulo <nombre>.  
 Entradas:  
 <tablas, rutas, pantallas, reglas>.  
 Entregables obligatorios:  
 1) plan breve,  
 2) archivos creados/modificados,  
 3) pruebas/manual QA,  
 4) riesgos,  
 5) siguientes pasos.  
 Restricciones:  
 No mezclar lógica de negocio en Blade.  
 No usar CSS inline ni JS inline.  
 No romper contratos existentes de API.  
 No tocar otros módulos sin justificarlo.

### 4. Arquitectura general recomendada

| Capa | Tecnología | Función | Regla principal |
|------|------------|---------|-----------------|
|------|------------|---------|-----------------|

|               |                                    |                                                                                   |                                               |
|---------------|------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------|
| Sitio público | WordPress                          | SEO, landing pages, formularios, campañas, contacto, WhatsApp y click-to-call     | No almacenar lógica operativa crítica         |
| Backoffice    | Laravel                            | Operación diaria, módulos de negocio, panel administrativo, validaciones y flujos | Controladores delgados y servicios explícitos |
| API           | Laravel API v1                     | Consumo por panel móvil, Android y automatizaciones                               | Versionar endpoints y respuestas              |
| Base de datos | PostgreSQL o MySQL                 | Persistencia central                                                              | Migrations, llaves, auditoría y estados       |
| App móvil     | Android / WebView-PWA o app nativa | Operación móvil del staff y consulta de estados                                   | Consumir la misma API                         |

La web pública y el sistema operativo no deben compartir la misma lógica interna. WordPress puede capturar leads o solicitudes, pero las reglas de agenda, hotel, ocupación, estados y recordatorios deben vivir en Laravel y su base de datos.

## 5. Estrategia de repositorios y estructura física

Se recomienda monorepo con carpetas por aplicación. El objetivo es que Antigravity pueda trabajar por subárboles concretos sin tocar todo el proyecto en cada iteración.

```

/project-root
  /apps
    /web-public-wp          # Tema hijo, assets y conectores para WordPress
    /backoffice-laravel    # Panel interno + API
    /android-app           # App Android o shell WebView/PWA
  /packages
    /design-tokens          # Colores, spacing, tipografía, variables compartidas
    /api-contracts         # Esquemas JSON, ejemplos y DTOs
    /shared-docs           # reglas, prompts, decisiones, ADRs
  /infra
    /nginx-apache
    /deploy
    /backups
  /docs
    /architecture
    /db
    /ux
    /qa

```

## 6. Estructura modular del backoffice Laravel

```

/app
  /Domain
  /Leads
  /Clients

```

```
/Pets
/SpaBookings
/HotelReservations
/Stays
/ServiceStatus
/Notifications
/Users
/Reports
/Http
  /Controllers/Web
  /Controllers/Api/V1
  /Requests
  /Resources
  /Middleware
/Services
/Repositories
/Policies
/Jobs
/Events
/Listeners
/resources
  /views
    /layouts
    /components
    /pages
    /leads
    /clients
    /pets
    /spa-bookings
    /hotel
    /stays
    /reports
  /js
    /core
    /modules
      /leads
      /clients
      /pets
      /spa-bookings
      /hotel
      /stays
      /notifications
  /css
    /base
    /layout
    /components
    /modules
      /leads
      /clients
      /pets
      /spa-bookings
      /hotel
      /stays
/routes
web.php
```

```
api.php
/database
  /migrations
  /seeders
  /factories
```

## 7. Separación correcta entre HTML, CSS, JS y PHP

### 7.1 HTML / Blade

- Blade solo presenta información y compone vistas.
- Nada de consultas SQL, lógica de negocio extensa o cálculos de estado complejos dentro de la vista.
- Usar layouts base, componentes reutilizables y parciales por módulo.
- Semántica HTML clara: header, nav, main, section, article, aside, footer, form, label, fieldset y table solo cuando realmente haya datos tabulares.

### 7.2 CSS

- No usar estilos inline, salvo casos extremadamente justificados.
- Separar base, layout, componentes y módulos.
- Usar nombres previsibles. Conviene una convención tipo BEM o prefijos por módulo: c-card, l-grid, m-pet-profile, m-booking-status.
- Centralizar colores, espaciados, sombras, radios y tipografía en design tokens.
- Evitar que un módulo modifique estilos globales fuera de su alcance.

### 7.3 JavaScript

- Usar ES Modules y cargar scripts por módulo o por página.
- Nada de mezclar lógica de interfaz con llamadas remotas sueltas en plantillas.
- Cada módulo debe tener init(), eventos, servicios API y manejo de errores.
- Si una pantalla no necesita JS, no cargarlo.

### 7.4 PHP

- Controladores delgados. Deben validar, delegar a servicios y devolver respuesta o vista.
- La lógica de negocio va en Services/Domain.
- Acceso a datos mediante repositorios o modelos encapsulados; no repetir consultas en cada controlador.
- Validaciones con FormRequest. Autorización con Policies. Respuestas API con Resources.
- Transacciones en operaciones críticas: check-in, check-out, reprogramación, bloqueo de cupo, confirmación de pago o cambios masivos de estado.

## 8. Módulos funcionales del negocio

| Módulo | Responsabilidad               | Pantallas mínimas            | API mínima          |
|--------|-------------------------------|------------------------------|---------------------|
| Leads  | Registrar origen de campañas, | Bandeja, detalle, conversión | /leads, /leads/{id} |

|                |                                                           |                                     |                     |
|----------------|-----------------------------------------------------------|-------------------------------------|---------------------|
|                | WhatsApp, llamada o formulario                            |                                     |                     |
| Clientes       | Dueño, contacto, notas y consentimiento                   | Listado, ficha, historial           | /clients            |
| Mascotas       | Perfil, raza, peso, temperamento, vacunas, fotos, alertas | Listado, ficha, historial           | /pets               |
| Citas SPA      | Agenda por servicio, groomer y franja                     | Calendario, detalle, reprogramación | /spa-bookings       |
| Reservas hotel | Reserva planeada y cupo estimado                          | Calendario, tablero de ocupación    | /hotel-reservations |
| Estancias      | Check-in real, check-out real, ocupación efectiva         | Ingreso, salida, tablero vivo       | /stays              |
| Estados        | Timeline operativo: recibido, baño, corte, listo, etc.    | Panel rápido, detalle               | /service-status     |
| Notificaciones | Recordatorios y logs de envío                             | Plantillas, cola, historial         | /notifications      |

## 9. Modelo de datos base

La base debe distinguir entre lo reservado y lo realmente ejecutado. Esa diferencia evita errores en hotel y agenda.

| Tabla               | Propósito                  | Claves principales                    | Notas                                        |
|---------------------|----------------------------|---------------------------------------|----------------------------------------------|
| clients             | Dueños y contactos         | id, phone, email                      | Puede tener varias mascotas                  |
| pets                | Ficha de mascota           | id, client_id                         | Incluye alertas y atributos clínicos básicos |
| services            | Catálogo SPA/hotel         | id, type                              | Baño, corte, hotel, combo, extra             |
| spa_bookings        | Reserva planeada de SPA    | id, pet_id, service_id, scheduled_at  | No equivale a servicio realizado             |
| hotel_reservations  | Reserva planeada del hotel | id, pet_id, start_at, end_at          | Consume cupo planeado                        |
| stays               | Ocupación real             | id, pet_id, check_in_at, check_out_at | Consume cupo real                            |
| service_status_logs | Evolución del servicio     | id, booking_id                        | Timeline y auditoría                         |
| notification_queue  | Mensajes pendientes        | id, channel, send_at                  | Preparado para WhatsApp/SMS/email            |
| campaign_sources    | Origen del lead            | id, source, medium                    | SEO, Meta Ads, orgánico, llamada             |
| audit_logs          | Trazabilidad               | id, actor_id, action                  | Indispensable en cambios de estado           |

- Una reserva de hotel puede cancelarse sin generar estancia.

- Una estancia puede extenderse sin cambiar la reserva original, pero debe quedar registrada.
- Los recordatorios se calculan desde reserva o estancia según el caso.
- Los estados operativos se guardan como bitácora; no solo como un campo actual.

## 10. Convenciones de API

- Versionar desde el inicio: /api/v1/...
- Respuestas JSON consistentes: data, meta, errors.
- No exponer estructuras internas de la base sin transformación.
- Usar ids numéricos o UUIDs de forma consistente.
- Manejar paginación, filtros por fecha, estado, encargado, tipo de servicio y cliente.
- Usar endpoints específicos para acciones de negocio críticas: confirmar, reprogramar, check-in, check-out, cambiar estado, marcar listo.

```
GET    /api/v1/spa-bookings
POST   /api/v1/spa-bookings
POST   /api/v1/spa-bookings/{id}/confirm
POST   /api/v1/spa-bookings/{id}/status
POST   /api/v1/hotel-reservations/{id}/check-in
POST   /api/v1/stays/{id}/check-out
GET    /api/v1/occupancy?date=2026-03-17
GET    /api/v1/pets/{id}/timeline
```

## 11. SEO y sitio público en WordPress

WordPress debe actuar como capa pública de marketing y captación. Su trabajo es atraer, informar y convertir; no administrar la operación completa.

- Cada servicio principal necesita URL propia, metadata única y contenidos útiles.
- Landing por campaña con formulario de lead y parámetros UTM persistidos.
- Botones claros de WhatsApp y llamada telefónica.
- Schema local/business cuando aplique.
- No duplicar datos críticos del backoffice dentro de WordPress.
- Cuando un formulario capture un lead, enviarlo por API o webhook al módulo de leads.

## 12. Estrategia Android

No conviene construir una app Android nativa completa desde el día uno si el flujo operativo todavía está evolucionando. La estrategia correcta es por fases.

| Fase   | Entrega                            | Propósito                                    |
|--------|------------------------------------|----------------------------------------------|
| Fase 1 | Panel responsivo / PWA             | Validar flujo real de staff y estados        |
| Fase 2 | Shell Android o WebView endurecido | Acceso rápido, icono, notificaciones básicas |

|        |                                           |                                                 |
|--------|-------------------------------------------|-------------------------------------------------|
| Fase 3 | App Android con módulos nativos puntuales | Cámara, notificaciones push, offline controlado |
|--------|-------------------------------------------|-------------------------------------------------|

La app Android debe consumir la misma API que el backoffice. Nunca debe tener reglas duplicadas para cálculo de cupo, estados o agenda.

## 13. Diseño UI y componentes

- Diseño sobrio, operativo y rápido. Menos decoración, más legibilidad.
- Tableros con métricas visibles: citas del día, check-ins esperados, mascotas listas, ocupación real, pendientes.
- Componentes base: cards, badges de estado, tablas filtrables, timeline, calendar slots, drawer lateral, toasts, modales de confirmación.
- Estados con color semántico consistente: pendiente, confirmado, en proceso, listo, cancelado.
- Diseño mobile-first para staff, desktop-first para administración extendida.
- Accesibilidad operativa: contraste suficiente, targets táctiles razonables, formularios claros.

## 14. Reglas de seguridad y calidad

- Autenticación y roles: recepción, groomer, encargado, administrador.
- CSRF, validación servidor, sanitización y rate limiting en formularios públicos.
- No guardar secretos en repositorio. Variables de entorno separadas.
- Backups de base de datos y de uploads.
- Logs funcionales y técnicos separados.
- Pruebas mínimas por módulo: rutas felices, errores esperados, permisos y datos límite.
- Checklist visual antes de publicar: móvil, tablet, escritorio, estados vacíos, formularios, tablas, filtros.

## 15. Flujo de trabajo recomendado

- Paso 1: definir módulo, objetivo, reglas y pantallas.
- Paso 2: diseñar tablas, relaciones, estados y endpoints.
- Paso 3: pedir a Antigravity plan de implementación con archivos afectados.
- Paso 4: construir backend primero; después vistas; después JS; después QA.
- Paso 5: validar con datos reales o seeders controlados.
- Paso 6: documentar decisión técnica y riesgos.
- Paso 7: solo entonces integrar con WordPress, Android o automatizaciones.

## 16. Checklist de definición por módulo

- Objetivo del módulo descrito en una frase.
- Actores que lo usan.
- Estados y transiciones.
- Tablas o cambios de base de datos.



- Endpoints requeridos.
- Pantallas y componentes UI.
- Validaciones de negocio.
- Permisos por rol.
- Eventos o notificaciones disparadas.
- Pruebas mínimas y riesgos.

## 17. Prompt maestro sugerido para Antigravity

```
Actúa como arquitecto y desarrollador senior.
Trabajarás solo dentro del módulo indicado.
Primero genera:
1) resumen de objetivo,
2) plan por pasos,
3) archivos a crear o modificar,
4) modelo de datos,
5) endpoints,
6) checklist de pruebas.
Luego implementa respetando estas reglas:
- HTML semántico.
- CSS modular sin estilos inline.
- JS modular por página o por feature.
- PHP con controladores delgados y lógica en servicios.
- Nada de SQL incrustado en vistas.
- No tocar otros módulos sin explicar por qué.
Al final entrega:
- diff lógico,
- pasos de prueba,
- riesgos,
- deuda técnica pendiente.
```

## 18. Conclusión

La mejor forma de construir este sistema en Antigravity no es pedirle una aplicación completa en un solo prompt. La forma correcta es trabajar por módulos pequeños, contratos claros, base de datos estable y entregables verificables. Si se respeta esta guía, el proyecto puede crecer desde un MVP de agenda hasta una plataforma operativa con hotel, estados, recordatorios, reportes y app móvil sin convertirse en un sistema frágil o difícil de mantener.

Siguiente paso recomendado: definir y aprobar el modelo de datos inicial y después construir el módulo de Leads, Clientes y Mascotas antes de tocar agenda o hotel.

## Anexo A. Referencias base para Antigravity

La lógica de trabajo de este manual asume el uso de Planning/Fast, Artifacts, Editor + Manager y controles de seguridad del agente. Estos conceptos provienen de la documentación pública de Antigravity y de sus codelabs oficiales.

- Getting Started with Google Antigravity
- Building with Google Antigravity

- Antigravity MCP integration
- Build with Google Antigravity, our new agentic development platform